

# Integer Programming Models

## 1 Introduction

We now begin the study of Integer Programs (IP), which are optimization problems of the form<sup>1</sup>

$$\begin{aligned} \max \quad & c^T x \\ Ax \quad & \leq b \\ x \quad & \in \mathbb{Z}^n. \end{aligned} \tag{1}$$

IPs are a much more powerful and realistic modeling tool than LPs. However, they are also harder to solve, both in theory<sup>2</sup> and in practice. This lecture is dedicated to introducing relevant problems that can be modeled via integer programming.

The most obvious way we can use integer variables is to require that the output solution is integer. This is of interest for, eg, the chair / table production problem we have seen multiple time through the class, since formulating it as a Linear Program does not guarantee the integrality of the solutions. However, there are less obvious ways integer variables can be used in modeling a problem. They often rely on the fact that a *Binary Integer Program* like the one below

$$\begin{aligned} \max \quad & c^T x \\ Ax \quad & \leq b \\ x \quad & \in \{0, 1\}^n \end{aligned} \tag{2}$$

is a special case of (1), since constraints

$$x \in \{0, 1\}^n$$

can be modeled as

$$0 \leq x \leq 1, x \in \mathbb{Z}^n.$$

In the following, we see many problems that can be modeled as Binary Integer Programs.

## 2 Combinatorial problems

In many problems, we want to choose a set  $S$  of minimum cost or maximum profit from a family of sets contained in a ground set  $V$ . Many such problems can be formulated as Binary Integer Programs.

<sup>1</sup>We are assuming constraints of the form  $Ax \leq b$  since every IP can be easily reduced to this form.

<sup>2</sup>In particular, there exists algorithms that solve Linear Programming in time polynomial in the size of the input, while no such algorithm is known for Integer Programming, which is NP-Hard.

## 2.1 Maximum Independent Set

An *independent set* of a graph  $G(V, E)$  is a set  $S \subseteq V$  such that no two nodes of  $S$  are *adjacent*<sup>3</sup> in  $G$ . The *Maximum Independent Set problem* (MIS) is defined as follows:

- Given: A graph  $G(V, E)$ ;
- Find: An independent set of  $G$  of maximum cardinality.

MIS has many applications. We list a few below:

- Event selection: We are attending a conference and have selected some talks that we would like to attend. However, many of them overlap in time. We can find the maximum number of non-overlapping talks by formulating a MIS whose vertices are the talks and two talks are adjacent if and only if they overlap in time.
- Frequency assignment: There are multiple stations that are willing to transmit on a given frequency. Some of them interfere with each other. We want to find the largest set of non-interfering stations that can transmit on the given frequency. We can solve this problem by finding a MIS where stations are vertices and two stations are adjacent if they interfere with each other.
- Task Assignment: Suppose you have  $n$  tasks; each task uses one or more of  $m$  resources. We want to find the largest set of tasks that can be performed concurrently, since they do not need the same resource. We can solve this problem by finding a MIS where tasks are vertices and two tasks are adjacent if they use the same activities.

To formulate MIS as an IP, we define a binary variable associated to every node  $i \in V$ . The idea is that an independent set  $S$  corresponds to a binary vector  $x$  such that

$$x_i = \begin{cases} 1 & \text{if } i \in S \\ 0 & \text{otherwise} \end{cases}.$$

We can now formulate the IP

$$\begin{aligned} \max \quad & \sum_{i \in V} x_i \\ & x_i + x_j \leq 1 \quad \text{for } ij \in E \\ & x \in \{0, 1\}^V. \end{aligned} \tag{3}$$

where the constraints are, on top of the one enforcing that  $x$  is a binary vector, those forbidding that two adjacent vertices are inserted in the set. The objective function just counts the number of non-zero entries in  $x$ , which is exactly the cardinality of  $S$ .

**Example.** Let  $G = (V, E)$  with  $V = \{1, 2, 3, 4\}$  and

$$E = \{(1, 2), (2, 3), (3, 4), (4, 1)\} \quad (\text{a 4-cycle}).$$

A largest independent set has size 2 (e.g.,  $\{1, 3\}$  or  $\{2, 4\}$ ). The IP formulation is

$$\begin{aligned} \max \quad & x_1 + x_2 + x_3 + x_4 \\ \text{s.t.} \quad & x_1 + x_2 \leq 1, \\ & x_2 + x_3 \leq 1, \\ & x_3 + x_4 \leq 1, \\ & x_4 + x_1 \leq 1, \\ & x_i \in \{0, 1\} \quad (i = 1, \dots, 4). \end{aligned}$$

<sup>3</sup>Two nodes are adjacent if they are connected by an edge.

## 2.2 Minimum Vertex Cover

Consider a graph representing, e.g., a computer network, with vertices being routers and servers, and edges being connections between them. Let us assume that intruders can attack connections and, when we monitor a server or router, we can detect an intrusion on all the connections incident to it. We want to find the minimum number of servers or routers to monitor to detect an intrusion in the network.

This problem can be formulated as an instance of the Minimum Vertex Cover. A *vertex cover* of a graph  $G(V, E)$  is a set  $S \subseteq V$  such that, for each edge  $ij \in E$ , either  $i$  or  $j$  (or both) belong to  $S$ . The Minimum Vertex Cover problem is the following:

- Given: A graph  $G(V, E)$ ;
- Find: A vertex cover  $S$  of  $G$  of minimum cardinality.

Following an argument similar to the one for MIS, a minimum vertex cover can be found by solving the IP

$$\begin{aligned} \min \quad & \sum_{i \in V} x_i \\ & x_i + x_j \geq 1 \quad \text{for } ij \in E \\ & x \in \{0, 1\}^V. \end{aligned}$$

**Example.** Use the same  $G = (V, E)$  as in the maximum independent set problem. A minimum vertex cover is given, e.g., by  $\{1, 3\}$ . The IP formulation of minimum vertex cover is:

$$\begin{aligned} \min \quad & x_1 + x_2 + x_3 + x_4 \\ \text{s.t.} \quad & x_1 + x_2 \geq 1, \\ & x_2 + x_3 \geq 1, \\ & x_3 + x_4 \geq 1, \\ & x_4 + x_1 \geq 1, \\ & x_i \in \{0, 1\} \quad (i = 1, \dots, 4). \end{aligned}$$

## 2.3 Minimum Dominating Set

Now consider a modification of the problem from the previous section, where an intruder can attack vertices of the network and, by monitoring a vertex, we will discover an intrusion in any vertex adjacent to it. Again, we want to find the minimum number of vertices to monitor to detect an intrusion in the network.

This problem can be formulated as an instance of the Minimum Dominating Set problem. A *dominating set* of a graph  $G(V, E)$  is a set  $S \subseteq V$  such that, for each vertex  $i \in V$ , there exists  $j \in S$  such that either  $i = j$ , or  $ij \in E$ . The Minimum Dominating Set problem is the following:

- Given: A graph  $G(V, E)$ ;
- Find: A dominating set of  $G$  of minimum cardinality.

Note that our IP for the Minimum Dominating Set problem will be a bit different from the ones before, since we can formulate the condition of  $S$  being a dominating set as a series of constraints on the nodes, not on the edges. We have:

$$\begin{aligned} \min \quad & \sum_{i \in V} x_i \\ & x_i + \sum_{j \in \delta(i)} x_j \geq 1 \quad \text{for } i \in V \\ & x \in \{0, 1\}^V. \end{aligned}$$

In the above, for a node  $i \in V$  we define  $\delta(i) = \{j \in V : ij \in E\}$  to be the set of all nodes adjacent to  $i$ .

**Example.** Let  $G = (V, E)$  be the path on five nodes:

$$V = \{1, 2, 3, 4, 5\}, \quad E = \{(1, 2), (2, 3), (3, 4), (4, 5)\}.$$

A minimum dominating set has size 2 (e.g.,  $\{2, 4\}$ ). The IP formulation is given by

$$\begin{array}{ll} \min & x_1 + x_2 + x_3 + x_4 + x_5 \\ \text{s.t.} & x_1 + x_2 \geq 1 \quad (\text{vertex 1}), \\ & x_1 + x_2 + x_3 \geq 1 \quad (\text{vertex 2}), \\ & x_2 + x_3 + x_4 \geq 1 \quad (\text{vertex 3}), \\ & x_3 + x_4 + x_5 \geq 1 \quad (\text{vertex 4}), \\ & x_4 + x_5 \geq 1 \quad (\text{vertex 5}), \\ & x_i \in \{0, 1\} \quad (i = 1, \dots, 5). \end{array}$$

## 2.4 Maximum Matching

We have already discussed matchings when we introduced the assignment game in a previous lecture. Recall that a matching of a graph  $G(V, E)$  is a set  $M \subseteq E$  such that there no edges  $e \neq f \in M$  share an endpoint. That is, there are no edges  $ij, i\ell \in M$  for some  $i, j, \ell$ . The Maximum Matching problem is the following.

- Given: A graph  $G(V, E)$ ;
- Find: A matching of  $G$  of maximum cardinality.

The Maximum Matching problem has many applications:

- Job/Workers assignment: Given jobs and workers who can perform them, find a way to assign the largest number of jobs to workers, under the condition that a worker must be assigned to a job they can do, that each worker can be assigned at most one job, and that each job can be assigned to at most one worker.
- Carpooling: Suppose that you have some riders who want to carpool, and assume that each car can host at most 2 of them. Some riders come and go to similar destinations, so they can carpool, while others don't. Create a graph where every node is a rider and two riders are adjacent if they can carpool. A maximum matching of this graph will tell you a strategy to maximize the number of riders that can share a ride.
- Kidney Exchange: Many patients with a kidney illness need a transplant. A loved one often volunteers for donating one of their kidneys (a healthy person can live with one kidney only), but the patient and the donor may not be compatible (because, e.g., of different blood types). Matching can help: create a graph where each vertex is a pair (patient, donor) and add an edge between a node (patient<sub>1</sub>, donor<sub>1</sub>) and (patient<sub>2</sub>, donor<sub>2</sub>) if the kidney of donor<sub>1</sub> is compatible with patient<sub>2</sub> and the kidney of donor<sub>2</sub> is compatible with patient<sub>1</sub>. A maximum matching will then give the maximum number of kidney transplants that can be performed by exchanging donors<sup>4</sup>.

We can formulate the Maximum Matching problem as follows:

<sup>4</sup>There are more complex models of kidney exchange, based on the fact that kidney can be shared along cycles. Such problems can be formulated via Integer Programming as well, even though they are not maximum matching problems.

$$\begin{aligned}
\max \quad & \sum_{e \in E} x_e \\
\sum_{j \in \delta(i)} x_{ij} & \leq 1 \quad \text{for } i \in V \\
x & \in \{0, 1\}^E.
\end{aligned} \tag{4}$$

Note that, unlike the previous problem, variables are defined over the edges (since a matching is a set of edges). The constraints model the fact that, for every node  $i$ , there can be at most one edge of the matching that contains  $i$ .

**Example:  $G$  bipartite.** A graph is *bipartite* if its nodes can be partitioned into two sets  $U, W$  so that there are only edges between nodes in  $U$  and nodes in  $W$ , but there are no edges between two nodes in  $U$ , or two nodes in  $W$ . Note that in the job / worker assignment problem from above, the graph is bipartite, while in the carpooling and kidney exchange problem in general it is not. Consider the bipartite graph  $G = (U \cup W, E)$  with

$$U = \{a, b, c\}, \quad W = \{1, 2, 3\}, \quad E = \{(a, 1), (a, 2), (b, 2), (b, 3), (c, 3)\}.$$

A maximum matching has size 3 (e.g.,  $\{(a, 1), (b, 2), (c, 3)\}$ ). The IP formulation is given by

$$\begin{aligned}
\max \quad & x_{a1} + x_{a2} + x_{b2} + x_{b3} + x_{c3} \\
\text{s.t.} \quad & x_{a1} + x_{a2} \leq 1 && (\text{vertex } a), \\
& x_{b2} + x_{b3} \leq 1 && (\text{vertex } b), \\
& x_{c3} \leq 1 && (\text{vertex } c), \\
& x_{a1} \leq 1 && (\text{vertex } 1), \\
& x_{a2} + x_{b2} \leq 1 && (\text{vertex } 2), \\
& x_{b3} + x_{c3} \leq 1 && (\text{vertex } 3), \\
& x_e \in \{0, 1\} \quad (\forall e \in E).
\end{aligned}$$

**Example:  $G$  non-bipartite.** Let  $G = (V, E)$  with

$$V = \{1, 2, 3, 4, 5\}, \quad E = \{(1, 2), (1, 3)(2, 3), (3, 4), (4, 5), (5, 1)\},$$

i.e., the 5-cycle  $C_5$ . This graph is *not* bipartite. A maximum matching has cardinality 2, e.g.,  $\{(1, 2), (3, 4)\}$  or  $\{(2, 3), (4, 5)\}$ . The IP formulation is

$$\begin{aligned}
\max \quad & x_{12} + x_{23} + x_{34} + x_{45} + x_{51} \\
\text{s.t.} \quad & x_{12} + x_{13} + x_{51} \leq 1 && (\text{vertex } 1), \\
& x_{12} + x_{23} \leq 1 && (\text{vertex } 2), \\
& x_{13} + x_{23} + x_{34} \leq 1 && (\text{vertex } 3), \\
& x_{34} + x_{45} \leq 1 && (\text{vertex } 4), \\
& x_{45} + x_{51} \leq 1 && (\text{vertex } 5), \\
& x_{ij} \in \{0, 1\} && \forall (i, j) \in E.
\end{aligned}$$

*Comment.* The degree constraints enforce that no two chosen edges share a vertex, so any feasible solution is a matching. The optimal objective value is 2.

## 2.5 Weighted versions

In some applications, instead of minimizing / maximizing the cardinality of a set, we want to minimize or maximize its weight. For instance, in the applications of MIS (Section 2.1), not all talks will have the same value for us, and we want to maximize the value of the selected talks. Similarly, in the job/worker application from Section 2.4, every pair

worker may be paid a different amount of money for a different job (depending on the job, the worker expertise on the job,...) and the goal may be to maximize the total salaries paid to workers. In such cases, we are also given in input a weight function  $w$  that associate to each node, or to each edge, a value. For instance, in MIS, we are given a weight function  $w : V \rightarrow \mathbb{Z}$  and we can formulate the new problem by replacing the objective function from (3) with

$$\max \sum_{i \in V} w_i x_i.$$

In the weighted version of the Maximum Matching problem, we are given a weight function  $w : E \rightarrow \mathbb{Z}$  and we can formulate the new problem by replacing the objective function from (4) with

$$\max \sum_{e \in E} w_e x_e.$$

The weighted version of the other problems seen so far is defined analogously.

**Example: Weighted MIS.** Consider again the graph used as an example for the Maximum Independent Set problem defined above. Let the node weights be  $w = (w_1, w_2, w_3, w_4) = (4, 2, 6, 5)$ . Then the optimal solution takes nodes 1 and 3. The IP formulation is

$$\begin{aligned} \max \quad & 4x_1 + 2x_2 + 6x_3 + 5x_4 \\ \text{s.t.} \quad & x_1 + x_2 \leq 1, \quad x_2 + x_3 \leq 1, \quad x_3 + x_4 \leq 1, \quad x_4 + x_1 \leq 1, \\ & x_i \in \{0, 1\} \quad (i = 1, \dots, 4). \end{aligned}$$

**Example: Weighted Matching.** Consider again the graph used as an example for the Maximum Matching problem defined above. Let edge weights  $w_{a1} = 7, w_{a2} = 4, w_{b2} = 5, w_{b3} = 6, w_{c3} = 3$ .

$$\begin{aligned} \max \quad & 7x_{a1} + 4x_{a2} + 5x_{b2} + 6x_{b3} + 3x_{c3} \\ \text{s.t.} \quad & x_{a1} + x_{a2} \leq 1, \quad x_{b2} + x_{b3} \leq 1, \quad x_{c3} \leq 1, \\ & x_{a1} \leq 1, \quad x_{a2} + x_{b2} \leq 1, \quad x_{b3} + x_{c3} \leq 1, \\ & x_e \in \{0, 1\} \quad (\forall e \in E). \end{aligned}$$

## 2.6 Graph Coloring

Consider the modification of the Frequency Assignment problem seen in Section 2.1. There are again multiple stations that are willing to transmit, with some of them interfering with each other. We want to find the minimum number of frequencies that we need to assign to stations such that each station is assigned a frequency, and no two stations that are assigned the same frequency interfere. This is an instance of the graph coloring problem, defined next.

- Given: Graph  $G(V, E)$
- Find: The smallest  $k \in \mathbb{N}$  such that the vertices of  $G$  can be colored with  $k$  colors so that no two adjacent vertices are assigned the same color.

Indeed, we can again think of every station as a vertex of  $G$ , with an edge between stations that interfere. A color is a frequency, and minimizing the number of used colors implies minimizing the number of frequencies that we will use.

To formulate the Graph Coloring Problem as an IP, we will use two sets of variables. The first,  $x \in \{0, 1\}^{V \times V}$  denotes which color is used for every node.

$$x_{ij} = \begin{cases} 1 & \text{if node } i \text{ is colored with color } j \\ 0 & \text{otherwise} \end{cases}.$$

The second set of variables,  $y \in \{0, 1\}^V$ , denotes the set of colors that are used.

$$y_j = \begin{cases} 1 & \text{if color } j \text{ is used, i.e., there exists some node colored with color } j \\ 0 & \text{otherwise} \end{cases}.$$

We can then formulate our IP as follows.

$$\begin{aligned} \min \quad & \sum_{j \in V} y_j \\ & \sum_{j \in V} x_{ij} = 1 \quad \text{for } i \in V \\ & x_{ij} + x_{\ell j} \leq y_j \quad \text{for } i\ell \in E \\ & x \in \{0, 1\}^{V \times V} \\ & y \in \{0, 1\}^V. \end{aligned} \tag{5}$$

In the IP above, the objective function aims at minimizing the number of colors used. The first set of constraints imposes that each vertex is colored with some color. To understand the last set of constraints, assume first  $y_j = 0$ . Then the constraints become  $x_{ij} + x_{\ell j} \leq 0$ , which together with nonnegativity implies  $x_{ij} = x_{\ell j} = 0$ . That is, since  $y_j = 0$ , we cannot color  $i$  or  $\ell$  with color  $j$ . Now suppose  $y_j = 1$ . Then the constraints becomes  $x_{ij} + x_{\ell j} \leq 1$  which, together with the binary constraint, implies that at most one of  $x_{ij}$  and  $x_{\ell j}$  is equal to 1. That is, at most one of  $i$  and  $\ell$  can be colored with color  $j$  (since they are adjacent).

**Example.** Let  $G = (V, E)$  with  $V = \{1, 2, 3, 4\}$  and

$$E = \{(1, 2), (2, 3), (3, 4), (4, 1), (2, 4)\}.$$

This graph contains triangle  $(1, 2, 4)$ , so it needs at least 3 colors. We provide a formulation with color index set  $K = \{1, 2, 3, 4\}$ . Variables  $x_{ij} \in \{0, 1\}$  indicate whether vertex  $i$  uses color  $j$ , and  $y_j \in \{0, 1\}$  indicate whether color  $j$  is used:

$$\begin{aligned} \min \quad & y_1 + y_2 + y_3 + y_4 \\ \text{s.t.} \quad & x_{11} + x_{12} + x_{13} + x_{14} = 1, \\ & x_{21} + x_{22} + x_{23} + x_{24} = 1, \\ & x_{31} + x_{32} + x_{33} + x_{34} = 1, \\ & x_{41} + x_{42} + x_{43} + x_{44} = 1, \\ & x_{1j} + x_{2j} \leq y_j \quad (j = 1, \dots, 4), \\ & x_{2j} + x_{3j} \leq y_j \quad (j = 1, \dots, 4), \\ & x_{3j} + x_{4j} \leq y_j \quad (j = 1, \dots, 4), \\ & x_{4j} + x_{1j} \leq y_j \quad (j = 1, \dots, 4), \\ & x_{2j} + x_{4j} \leq y_j \quad (j = 1, \dots, 4), \\ & x_{ij} \in \{0, 1\} \quad (i = 1, \dots, 4; j = 1, \dots, 4), \\ & y_j \in \{0, 1\} \quad (j = 1, \dots, 4). \end{aligned}$$

### 3 Other usage of binary variables

We now discuss some other uses of binary variables in modeling. Many of the resulting problems will be of *mixed-integer* type. That is, some of the variables will be integer, while others are allowed to be continuous. Mixed-integer problems are often treated together with (pure) Integer Programs in the literature<sup>5</sup>. Thus, by abusing notation a bit, we are still going to call them Integer Programs.

<sup>5</sup>Because, for instance, the algorithms for their solutions are analogous.

### 3.1 Fixed and setup costs

A fixed/setup cost associated to a variable  $x \geq 0$  is of the form

- If  $x = 0$  there is no cost;
- Else if  $x > 0$  we pay a *fixed/setup* cost  $s$ , plus a cost that is linear in  $x$  and is given by  $c \cdot x$ . We assume  $c, s > 0$ .

Such a scenario can be modeled by introducing a variable  $z \in \{0, 1\}$  and writing the constraint

$$x \leq Mz, \quad (6)$$

where  $M$  is a number that is assumed to be an upper bound to  $x$  (that is,  $x \leq M$  for every feasible solution<sup>6</sup>). The cost associated to  $x$  in the objective function will be

$$c \cdot x + s \cdot z.$$

Let us now see why this formulation is correct. When  $z = 0$ , constraint (6) and  $x \geq 0$  forces  $x$  to be equal to 0. In this case, the cost in the objective function is also equal to 0, as required. Now assume  $x > 0$ . Then we must have  $z = 1$  (because of the argument above) and consequently we are going to pay  $c \cdot x + s$  as a cost, as required. Note that there is a third case, namely  $x = 0$  and  $z = 1$ , but this will not be an optimal solution, since an optimal solution prefers resetting  $z = 0$  to avoid incurring in the cost  $s$ .

### 3.2 Either / Or constraints

Suppose that we have a constraint saying that exactly one of two situations must happen, for instance:

- In a production problem, either we do not produce any compact car, or we produce at least 1000 of them;
- In a logistic problem, either we don't ship any product between warehouse  $i$  and shop  $j$ , or we ship at least 1000 units.

In formulas, letting  $y \geq 0$  be the variable denoting cars / shipments in the examples above, we have

$$y = 0 \quad \text{or} \quad y \geq 1000.$$

We can model this scenario with a binary variable  $z \in \{0, 1\}$  and constraints

$$y \leq Mz; \quad y \geq 1000z,$$

where  $M$  is, here and in the rest of the section, a valid upper bound to the variable  $y$  (and to the other variables). When  $z = 0$ ,  $y$  is forced to be 0; conversely, when  $z = 1$ ,  $y$  is forced to be at least 1000, as required.

Let us consider a different constraint with a similar flavor: for variables  $x_1, x_2 \geq 0$

- at most one of  $x_1$  and  $x_2$  can be  $> 0$ .

We can model the situation above with a binary variable  $z \in \{0, 1\}$ :

$$x_1 \leq Mz; \quad x_2 \leq M(1 - z).$$

Let us consider a third case: for variables  $x_1, x_2, x_3 \geq 0$ ,

<sup>6</sup>If this number is not known, language programs often have pre-registered “very large numbers” such as the maximum integer a computer can store. While the IP formulation will be correct, if we take too large of a number the solution of the formulation will be harder.



- at most one of  $x_1, x_2$ , and  $x_3$  can be  $> 0$ .

We can model this scenario with three binary variable  $z, z', z'' \in \{0, 1\}$ :

$$z + z' + z'' = 1 \quad x_1 \leq Mz \quad x_2 \leq Mz' \quad x_3 \leq Mz''.$$

### 3.3 Piecewise linear objective functions

A function  $f$  is piecewise linear if there exist a finite set of intervals  $[-\infty, a_1), [a_1, a_2), [a_2, a_3), \dots, [a_k, +\infty)$  such that  $f$  is linear on each of them. Piecewise linear function can be used to model, for instance, economies of scale: if we buy at most  $a_1$  units of a product, we pay each unit  $c_1$ ; if we buy additional units, we pay for each unit between  $a_1$  and  $a_2$  a cost  $c_2 < c_1$ ; and for each additional unit a cost  $c_3 < c_2$ , etc.

Let us see an example of 3 pieces, from which one can easily abstract out the general case. Let

$$c = \begin{cases} 10 & \text{for each of the first 40 units} \\ 7 & \text{for each additional unit between 41 and 60} \\ 5 & \text{for each additional unit} \end{cases}$$

be the cost of a variable  $x \geq 0$ . To model the piecewise cost of  $x$ , we

- Add variables  $y_b, y_c \in \{0, 1\}$  and  $x^a, x^b, x^c \in \mathbb{R}_{\geq 0}$ ;
- In the objective function, replace  $10x$  with  $10x^a + 7x^b + 5x^c$ ;
- Add constraints:

$$\begin{aligned} x &= x^a + x^b + x^c \\ 0 &\leq x^a \leq 40 \\ 40y_b &\leq x^a \\ 0 &\leq x^b \leq 20y_b \\ 0 &\leq x^c \leq My_c \\ 20y_c &\leq x^b \end{aligned}$$

where again  $M$  is a valid upper bound<sup>7</sup> to  $x$ . Each of  $x^a, x^b$ , and  $x^c$  represents a “piece” of the variable  $x$  (respectively, between 0 and 40, between 40 and  $60 = 40 + 20$ , and after 60).

To verify that the constraints are correct, observe that the fourth and the sixth constraint above imply  $y_c \leq y_b$ . So it cannot happen that  $y_c = 1$  and  $y_b = 0$ . The possible cases are therefore:

- $y_b = y_c = 0$ . In this case, the constraints force  $x^b = x^c = 0, 0 \leq x^a \leq 40$ . Thus,  $x = x^a$  and we are in the first “piece” of the function.
- $y_b = 1, y_c = 0$ . In this case, the constraints force  $x^a = 40, 0 \leq x^b \leq 20, x^c = 0$ . Thus,  $x = 40 + x^b$ , and we are in the second “piece” of the function.
- $y_b = y_c = 1$ . In this case, the constraints force  $x^a = 40, x^b = 20, x^c \geq 0$ . Thus,  $x = 40 + 20 + x^c = 60 + x^c$ , and we are in the third and last “piece” of the function.

<sup>7</sup>Actually, it suffices that  $M$  is an upper bound to  $x - 60$ .